Books and spells

Status: Stage A, implemented. A book is a package of logic definitions. A spell is a book with a `main` definition, so the same source is automatically a CLI tool when it has an entry and a library when it does not. Prose no longer contains function definitions; it casts books. The example suite is 1530 checks (1488 examples, 42 properties), the granular suite is 33 cases, and discovery is 91 tests.

The idea

A child writes a tool once, in one file, and uses it two ways:

- as a **book** (a library), called directly by the IR code it contains;
- as a **spell** (a CLI tool), called through the shell capability or the spell runner.

The difference is not a declaration. A book becomes a spell when it defines `main`. That is the CLI logic. Without `main`, it is automatically a book.

The book source

A `.book` file contains definitions in the same expression language as the prose surface. `logic` is the ordinary definition marker; `cycle` is the recursive marker, and `and` groups cycle definitions F#-style:

```
logic double x = multiply(x 2).
logic quad x = double(double(x)).
cycle even n = odd(n).
and odd n = even(n).
```

A book has no free sentences; a sentence belongs in a `.prose` file. A book with no definitions is refused. A self, forward, or mutual call is an ordinary unit call. A `logic` body must not call its own name; a `cycle` body may call any name in its cycle group. The marker is checked against the calls, so a bad marker is a named refusal.

Each definition becomes a package unit. The package name is the file stem, version `1`, and the source hash is the SHA-256 of the file. Every definition is an export. `main` is the CLI entry.

Recursion

A book is one module group. All definitions are available to every body, in definition order, so `fact` and `even`/`odd` work through the group. A self, forward, or mutual call is an ordinary unit call; D188′s refusal is retired.

Each definition lowers to a unit with explicit arity and no lexical captures. A call resolves through the group, frames its arguments, and runs the unit. `MaxCallDepth` bounds a runaway group; exceeding it is the named `call depth exceeded: 256`.

The markers are diagnostics, not a second semantics. `logic` is an ordinary definition and refuses a body that calls its own name. `cycle` is recursive and stores its group with `and`; a `cycle` that calls no book word is refused. The plain word name is the recursive reference; there is no `recurse` word.

An F# Prelude function called by a book is compiled F# code; its own `let rec` is resolved by the F# compiler and is not affected by the book binding. The loader runs the requested definition, not the whole definition chain.

Classification

Source	Kind
`main` is defined	Spell: a CLI tool
`main` is absent	Book: a library

The classification is derived; there is no `book` or `spell` declaration to maintain.

Running a spell

```
scale --book tool.book
```

The loader prints the classification and the exports. A spell runs `main`; the final value is printed and the process exits with the last command status. A book prints its exports and exits zero. `main` is zero-argument in Stage A; command-line arguments to a spell are the next stage.

Casting a book

From the CLI:

```
scale --cast math.book double 21
```

From prose:

```
cast("math.book" "double" 21).
```

Both load the book and call the named function with the supplied arguments. A prose `cast` argument may be a number or a text value already on the stack; the CLI turns a numeric argument into a number and keeps every other argument as text. `cast("tool.book")` with only a path runs `main` when the package is a spell and is refused when it is a book.

A cast is a call, not a shell invocation. A relative path is resolved against the calling `.prose` file's directory; a book's nested casts resolve against that book's directory.

Prose has no logic

`logic` in a `.prose` file is `LogicOutsideBook`: a definition belongs in a `.book` file, and prose casts books instead. This supersedes D177 for the prose surface. The keyword remains reserved so it cannot become a name.

Refusals

- a missing book file, or a file with no definitions;
- a free sentence in a book file;
- an unknown cast word;
- a cast of a book without a word;
- a non-text path or word;
- a `logic` body that calls its own name;
- a `cycle` definition that calls no book word;
- a definition that cannot be packaged (an unsupported expression);
- a spell `main` with parameters;
- a call whose argument count does not match the definition.

Every refusal is a named value and a non-zero process status; none is a silent success.

Package contract

A book compiles to a `PackageAst.PackageDocument`:

Book	Package
file stem	name
`1`	version
SHA-256 of the file	package source hash
each definition	a unit whose root is the body or its lambda

Book	Package
each definition	a unit source hash from its portable definition
each definition	dependencies from the book words it calls
each definition	an export from the name to its unit
all definitions	one unit group with `InitOrder` and classification
`main`	the CLI entry

The package document is the only module format. The `.book` source is its front-end; there is no second serialization for books.

Loading is lazy: a book is parsed and packaged, but its definitions are not evaluated until they are called. A call checks the definition's arity before the image runs. A spell's `main` must take no arguments; the CLI supplies none. `MaxCallDepth` bounds recursive calls, and a group's cycle is ordinary recursion, not a special call path. An unchanged resolved path and source hash are served from the derived cache; `clearCache` resets it.

`open math` imports the book `math.book` beside the including file. Its exports become callable words in the importer, its package capabilities are unioned into the importer, and its package name/version is recorded in the importer's `Imports`. The open has no quotes and no `.book` suffix. A missing open, a duplicate open, an open cycle, and a conflicting definition name are named refusals. A book with opens is rebuilt when loaded, so a changed opened book cannot leave a stale parent.

The ordered open block at the top of the main book is its **project index**. There is no separate index file. Each opened file is listed once, in dependency order; a dependency must appear before the file that uses it. A missing dependency, duplicate entry, out-of-order entry, and cycle are named refusals. `BookRunner.index` returns the ordered entries and their dependencies. The index is deliberately local to the book project; a separate solution or project file is only needed when one repository has several roots.

A **scroll** is a book without an index: definitions only, no opens, no project ordering. A **directory** orders the scrolls and books of one library with `include` entries; it has no `main`. The **catalog** is the workspace-wide lookup of libraries, projects, and scrolls across Polylith. A catalog entry carries a name, kind, path, version, and capabilities. The catalog is deliberately not a global build order: dependencies and cycles stay local to each project or library index.

Host words are effect ports. Each adapter declares the capability it provides; the run starts with no granted capabilities, and an attached adapter grants its own capability. Dispatch refuses a host word whose adapter capability is not granted. A book derives its required capabilities from the host words in its definitions, unions them with opened books, and records them in the package and image. A run checks the package's required capabilities before executing any definition. `--check PATH` is the non-running gate: it loads, packages, lowers, and checks `main` arity without executing a definition.

Testing

1. a definitions-only book parses without a trailing sentence;
2. a definitions-only book parses without a trailing sentence;
3. a book with `main` classifies as a spell; one without classifies as a book;
4. a free sentence in a book is refused;
5. prose refuses `logic` and `cycle` with `LogicOutsideBook`;
6. `cast("math.book" "double" 21)` returns 42;

7. a book word can call another definition;
8. a forward call runs;
9. a runaway recursive book is depth-bounded;
10. a `logic` self-call and a `cycle` without a book call are refused;
11. a spell's `main` runs and its last command status is the process status;
12. an unknown cast word is a named refusal;
13. a missing book is a named refusal;
14. the CLI selects `--book` and `--cast`;
15. the package document round-trips and exposes the definitions.
16. **Stage A.** The `.book` source, automatic recursive groups, optional markers, package classification, `--book`, `--cast`, the prose `cast` host word, and the prose `logic` refusal.
17. **Stage B.** Spell command-line arguments, nested book imports, and per-definition source hashes for hot reload.
18. **Stage C.** A spell executable (a book packaged with its runner) and capability grants for untrusted books.